Reliability & Disaster Recovery

> **Document Owner:** Platform Team | **Status:** Approved | **Last Updated:** February 2026

Overview

This document defines reliability, high availability, and disaster recovery patterns for the Databricks platform. These patterns ensure business continuity during failures, outages, and regional disruptions.

Golden Rules

ID	Rule	Severity	WAF Pillar
REL-01	Enable Delta Lake time travel (≥7 days retention)	Critical	Reliability
REL-02	Configure job retry with exponential backoff	Required	Reliability
REL-03	Use structured streaming with checkpointing	Critical	Reliability
REL-04	Enable cluster autoscaling for variable workloads	Required	Reliability
REL-05	Configure auto-termination on all clusters	Required	Reliability
REL-06	Use cluster pools for faster recovery	Recommended	Reliability
REL-07	Implement workspace backup procedures	Critical	Reliability
REL-08	Test disaster recovery procedures quarterly	Required	Reliability
REL-09	Use managed model serving for production ML	Required	Reliability
REL-10	Monitor and manage platform service limits	Recommended	OpEx
REL-11	Invest in capacity planning for production workloads	Recommended	OpEx

---

REL-01: Delta Lake Time Travel

Rule

All Delta tables must retain at least 7 days of history for point-in-time recovery.

Why It Matters

- Enables recovery from accidental deletes or updates
- Provides audit trail for compliance
- Supports debugging production issues
- No external backup system required

Implementation

```sql
-- Check current retention
DESCRIBE HISTORY catalog.schema.my_table;

-- Set retention (default is 30 days)
ALTER TABLE catalog.schema.my_table
SET TBLPROPERTIES ('delta.logRetentionDuration' = '7 days');

-- Restore to previous version
RESTORE TABLE catalog.schema.my_table TO VERSION AS OF 42;

-- Restore to timestamp
RESTORE TABLE catalog.schema.my_table TO TIMESTAMP AS OF '2025-02-01
00:00:00';
```

REL-02: Job Retry Configuration

Rule

Configure intelligent retry policies with exponential backoff for all production jobs.

Why It Matters

- Automatic recovery from transient failures
- Prevents overwhelming downstream services
- Distinguishes recoverable vs. permanent failures
- Reduces manual intervention

Implementation

```yaml
resources:
  jobs:
    production_etl:
      name: "[${bundle.target}] Production ETL"

      # Timeout at job level
      timeout_seconds: 7200  # 2 hours

      tasks:
        - task_key: process_data
          # Task-level retry configuration
          retry_on_timeout: true
          max_retries: 3
          min_retry_interval_millis: 60000  # 1 minute initial wait

          notebook_task:
            notebook_path: ../src/process.py
```

Retry Decision Guide

Error Type	Retry?	Max Attempts
Network timeout	Yes	3
Resource unavailable	Yes	3
Authentication failure	No	0
Data validation error	No	0
Out of memory	Maybe	1 (with larger cluster)

REL-03: Streaming Checkpointing

Rule

All structured streaming jobs must use checkpointing with highly available storage (ZRS or cross-region).

Why It Matters

- Enables exactly-once processing guarantees
- Allows automatic recovery from cluster failures
- Preserves processing state across restarts
- Supports seamless upgrades

Implementation

```
# Correct: ZRS storage for checkpoints
checkpoint_path =
"abfss://container@storage.dfs.core.windows.net/checkpoints/my_stream"

streaming_query = (
    spark.readStream
    .format("delta")
    .table("bronze.events")
    .writeStream
    .format("delta")
    .option("checkpointLocation", checkpoint_path)
    .trigger(availableNow=True)  # Or processingTime="10 seconds"
    .toTable("silver.events")
)

# Wait for completion (batch mode)
streaming_query.awaitTermination()
```

Checkpoint Storage Requirements

Feature	Requirement

Feature	Requirement
Storage type	Zone-redundant (ZRS) or geo-redundant (GRS)
Latency	Low latency for frequent writes
Durability	11 nines (99.999999999%)
Location	Same region as cluster

REL-04: Cluster Autoscaling

Rule

Enable autoscaling for all variable workloads. Use fixed sizing only for streaming with predictable load.

Why It Matters

- Handles demand fluctuations automatically
- Prevents under-provisioning during peak
- Optimizes cost during low demand
- No manual scaling intervention needed

Implementation

```
# For Serverless Jobs (automatic)
environments:
  - environment_key: default
    spec:
      environment_version: "4"  # Auto-scales automatically

# For Classic Clusters (when required)
new_cluster:
  autoscale:
    min_workers: 2
    max_workers: 10
  autotermination_minutes: 30
```

Autoscaling Guidelines

Workload	Autoscaling	Min Workers	Max Workers
Interactive notebooks	Yes	1	4
Variable batch ETL	Yes	2	10
Streaming (variable)	Limited	2	8
Streaming (steady)	No (fixed)	4	4
ML Training	No (fixed)	2	2

REL-05: Auto-Termination

Rule

Configure auto-termination on all clusters to prevent idle resource costs and ensure cluster refresh.

Why It Matters

- Reduces costs from idle clusters
- Forces regular cluster refresh (security patches)
- Prevents zombie clusters
- Maintains cluster hygiene

Settings by Environment

Environment	Auto-Terminate
Development	15 minutes
Staging	30 minutes
Production (batch)	60 minutes
Production (streaming)	Disabled (0)

REL-06: Cluster Pools

Rule

Use cluster pools for frequently used configurations to reduce startup time and improve availability.

Why It Matters

- Reduces startup from 5-10 minutes to <60 seconds
- Ensures instance availability during capacity constraints
- No DBU charges for idle pool instances
- Provides consistent cluster provisioning

Implementation

```
resources:
  cluster_pools:
    data_engineering_pool:
      pool_name: "[${bundle.target}] Data Engineering Pool"
      node_type_id: "i3.xlarge"
      min_idle_instances: 2
      max_capacity: 20
      idle_instance_autotermination_minutes: 60
      preloaded_spark_versions:
        - "14.3.x-scala2.12"
```

REL-07: Workspace Backup

Rule

Implement automated backup procedures for all workspace assets with cross-region replication.

Why It Matters

- Enables rapid recovery from workspace corruption
- Preserves development work and configurations
- Supports disaster recovery requirements
- Maintains audit trail

Backup Scope

Asset	Backup Method	Frequency
Notebooks	Git repos	Continuous
Job definitions	Asset Bundles	On deploy
Cluster configs	Asset Bundles	On deploy
Secrets	External vault	Daily
Unity Catalog metadata	System tables	Automatic

Implementation

```
# Export workspace artifacts using Databricks CLI
databricks workspace export_dir /Workspace/Shared ./backup/workspace/

# Export jobs
databricks jobs list --output JSON > ./backup/jobs.json

# For Asset Bundles - all configs are in Git
git push origin main  # Continuous backup
```

Failure Mitigation Patterns

Cluster Driver Node Failure

Mitigation	Implementation
Auto-restart policies	Enable on cluster config
Checkpointing	All streaming jobs

Mitigation	Implementation
Fault-tolerant state	Structured streaming
Idempotent operations	MERGE with deduplication

Job Execution Failures

Mitigation	Implementation
Retry with backoff	Task-level retry config
Timeout settings	Job and task level
Error handling	try/except with logging
Alerting	Email notifications

Data Corruption

Mitigation	Implementation
ACID transactions	Delta Lake (automatic)
Time travel	7+ day retention
DLT expectations	Quality enforcement
Validation checks	Pre-merge validation

Workspace Unavailability

Mitigation	Implementation
Multi-region deployment	For mission-critical
Git-based code	All code in repos
Asset Bundles	Declarative configs
Cross-region data replication	Delta Sharing or copy

REL-09: Managed Model Serving for Production ML

Rule

Use Databricks Model Serving (Mosaic AI) for all production ML model deployments. Avoid self-managed serving infrastructure.

Why It Matters

- Auto-scaling handles traffic spikes without manual intervention
- Built-in monitoring and logging via inference tables
- High availability with automatic failover

- Integrated with Unity Catalog for model governance

Implementation

```
import mlflow

# Deploy model from Unity Catalog to serving endpoint
client = mlflow.tracking.MlflowClient()

endpoint_config = {
    "served_entities": [{
        "entity_name": "catalog.schema.my_model",
        "entity_version": "1",
        "scale_to_zero_enabled": True,
        "workload_size": "Small"
    }]
}
```

REL-10: Monitor and Manage Platform Service Limits

Rule

Proactively monitor and manage Databricks platform service limits and quotas to prevent production disruptions.

Why It Matters

- Service limits (e.g., max clusters, max jobs, API rate limits) can silently block production workloads
- Approaching limits without awareness leads to unexpected failures
- Proactive monitoring enables timely limit increase requests

Implementation

```
-- Monitor active cluster count vs limits
SELECT
    workspace_id,
    COUNT(*) as active_clusters,
    100 as default_cluster_limit  -- Check actual limit in account
settings
FROM system.compute.clusters
WHERE state = 'RUNNING'
GROUP BY workspace_id;
```

Key Limits to Monitor

Resource	Default Limit	Monitor Via

Resource	Default Limit	Monitor Via
Concurrent clusters	100 per workspace	system.compute.clusters
Concurrent jobs	1000 per workspace	system.workflow.jobs
API rate limit	Varies by endpoint	Response headers
Warehouse concurrency	Varies by size	system.query.history

REL-11: Capacity Planning for Production Workloads

Rule

Invest in capacity planning for production workloads by analyzing historical usage patterns and projecting future resource needs.

Why It Matters

- Prevents under-provisioning during peak periods
- Supports budget forecasting and cost optimization
- Enables proactive infrastructure scaling
- Reduces risk of production failures from resource exhaustion

Implementation

```sql
-- Analyze usage growth trends for capacity planning
SELECT
    DATE_TRUNC('month', usage_date) AS month,
    sku_name,
    SUM(usage_quantity) AS total_dbus,
    SUM(list_cost) AS total_cost,
    LAG(SUM(usage_quantity)) OVER (PARTITION BY sku_name ORDER BY
DATE_TRUNC('month', usage_date)) AS prev_month_dbus,
    ROUND((SUM(usage_quantity) - LAG(SUM(usage_quantity)) OVER (PARTITION
BY sku_name ORDER BY DATE_TRUNC('month', usage_date)))
        / NULLIF(LAG(SUM(usage_quantity)) OVER (PARTITION BY sku_name
ORDER BY DATE_TRUNC('month', usage_date)), 0) * 100, 1) AS growth_pct
FROM system.billing.usage
WHERE usage_date >= current_date() - INTERVAL 6 MONTHS
GROUP BY 1, 2
ORDER BY month DESC, total_cost DESC;
```

Capacity Planning Checklist

Dimension	What to Plan	Data Source
Compute	Peak concurrent clusters, DBU consumption	system.billing.usage
Storage	Delta table growth rate	system.storage

Dimension	What to Plan	Data Source
Queries	Peak concurrent queries per warehouse	system.query.history
Jobs	Job count growth, failure rates	system.workflow.jobs

Disaster Recovery Procedures

RTO/RPO Targets

Tier	RTO	RPO	Use Case
Tier 1	<1 hour	<15 min	Critical pipelines
Tier 2	<4 hours	<1 hour	Standard ETL
Tier 3	<24 hours	<4 hours	Development

DR Runbook

1. **Detection**
    - Monitor workspace health via Azure Monitor
    - Automated alerts on job failures
    - Unity Catalog accessibility checks
2. **Assessment**
    - Determine scope of impact
    - Identify affected workloads
    - Check data integrity
3. **Recovery**
    - Restore workspace from backup (if needed)
    - Redeploy Asset Bundles
    - Restart streaming jobs from checkpoints
    - Validate data consistency
4. **Validation**
    - Run data quality checks
    - Verify job executions
    - Confirm downstream dependencies

Validation Checklist

Data Protection

- [ ] Delta Lake time travel ≥7 days retention
- [ ] Streaming checkpoints on ZRS/GRS storage

- [ ] DLT expectations for data quality
- [ ] MERGE operations are idempotent

Compute Resilience

- [ ] Autoscaling enabled for variable workloads
- [ ] Auto-termination configured
- [ ] Cluster pools for frequently used configs
- [ ] Job retry policies configured

Platform Health

- [ ] Service limits monitored with alerts (REL-10)
- [ ] Capacity planning review conducted quarterly (REL-11)

Operational Readiness

- [ ] Workspace backup procedures documented
- [ ] DR runbook tested quarterly
- [ ] Monitoring and alerting configured
- [ ] Recovery contacts documented

Related Documents

- Platform Overview
- Serverless Compute
- UC Tables & Delta Lake

References

- Reliability Best Practices
- Architecture Best Practices for Azure Databricks
- Structured Streaming Checkpointing
- Delta Lake Time Travel